MASTERCLASS · IDT

De carpeta vacía a tienda online.

Sin escribir una línea de código. Dirigiendo agentes de IA.

Acner Pinazo · Octubre 2026

En 2 horas vamos a construir:

Una tienda

React, Tailwind, Node y MySQL

Un equipo de IA

Agentes que trabajan en paralelo

Un Jarvis

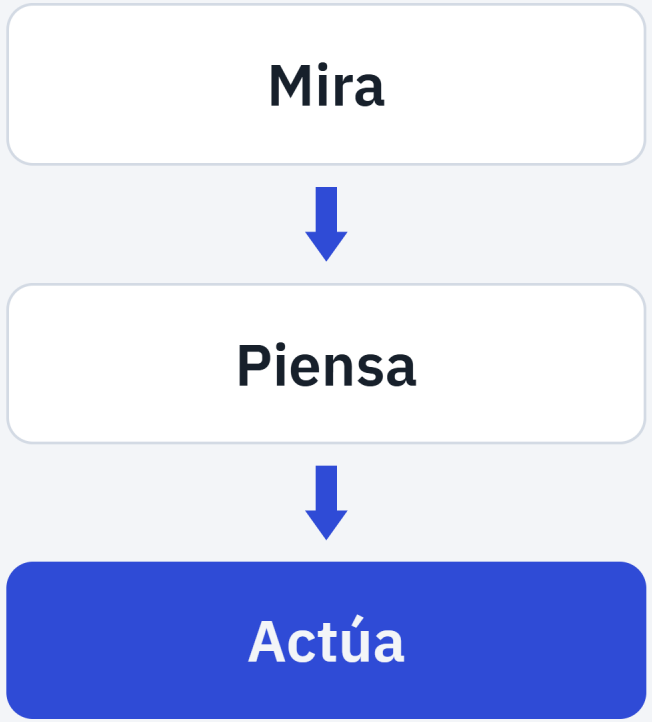
Para ver qué hace cada uno

Y lo vamos a controlar desde el celular.

CONCEPTO 1 · AGENTE

Un chat responde. Un agente hace.

- El chat: pregunto, responde, copio y pego.
- El agente: mira, piensa, actúa y vuelve a mirar, hasta terminar.
- El modelo no ejecuta nada: pide una herramienta y el programa la ejecuta. Eso es «tool use».
- ¿Quién ya vio a Antigravity crear archivos solo?



CONCEPTO 2 · CONTEXTO

La IA es un genio con amnesia.

- Cada conversación empieza de cero: no sabe tu proyecto ni tus reglas.
- Sólo sabe lo que tiene «sobre el escritorio». Eso es el contexto.
- El escritorio tiene un límite: si se llena, se olvida.
- Darle lo justo es el trabajo nuevo: context engineering.
- **CLAUDE.md** es su carpeta de onboarding: la lee siempre.



Claude Code o Antigravity: mismas ideas

Claude Code

Extensión de VS Code.
Lo usamos hoy porque deja ver todo lo que pasa adentro, y se maneja desde el celular.

Antigravity

El que ya usan.
Agent Manager, navegador propio, modelos Gemini y Claude.

Skills y MCP son estándares abiertos: lo de hoy sirve en los dos.

EN VIVO 1

Carpeta vacía y CLAUDE.md

1. Compartí VS Code.
2. Archivo › Abrir carpeta › **tiendita** (vacía).
3. Abrí el panel de Claude Code.
4. Escribí el prompt de la derecha.
5. Antes de aceptar, mostrá el pedido de permiso.
6. Aceptá y abrí **CLAUDE.md**.

ESCRIBÍ EN CLAUDE CODE

Vamos a crear una tienda online: frontend React + Tailwind en /frontend, backend Node + Express en /backend y MySQL en Docker (contenedor tienda-mysql, base tienda, usuario root, clave root). Escribí un CLAUDE.md con ese stack y estas reglas: precios en guaraníes enteros, SQL siempre con parámetros, validar con Zod.

Clave: no escribe nada sin mi permiso. En un proyecto real, nunca «aprobar todo».

EN VIVO 2

Primero el plan, después el código

1. **Shift + Tab** hasta que diga modo plan.
2. Escribí el prompt.
3. Leé el plan en voz alta y aprobalo.
4. Aprobá cada comando diciendo qué hace.
5. Pedí: «Levantá backend y frontend».
6. Abrí **localhost:5173** en el navegador.

ESCRIBÍ EN CLAUDE CODE

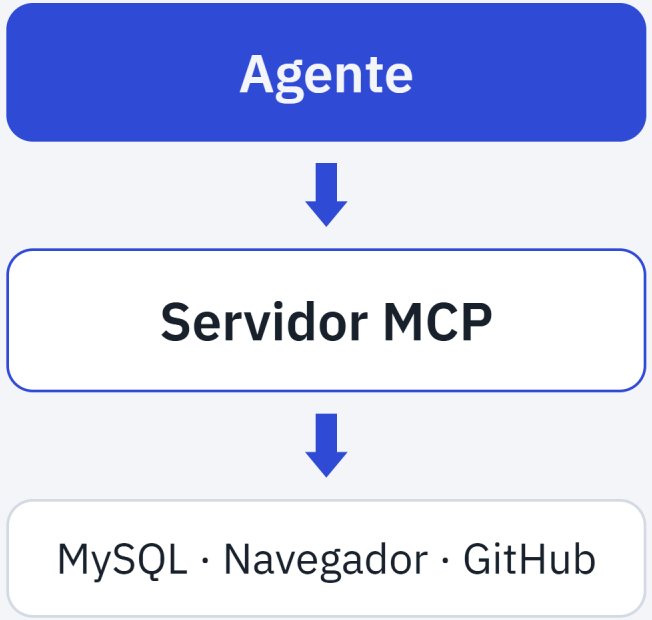
Armá el proyecto completo: la base de datos con productos paraguayos de ejemplo (tereré, mates, yerba, accesorios), la API de productos y una página con la grilla de productos y un carrito. Mostrame el plan antes de tocar nada.

Clave: tarda 5 a 8 minutos, momento de preguntas. No escribí código, pero tomé todas las decisiones.

CONCEPTO 3 · MCP

MCP: el USB-C de la IA.

- Si le pregunto por el stock sin acceso a la base, inventa. Y con mucha seguridad.
- MCP es un enchufe estándar: base de datos, navegador, GitHub, la API de tu empresa.
- El servidor MCP ofrece herramientas; el agente las pide cuando las necesita.
- **Sin MCP inventa. Con MCP pregunta.**



EN VIVO 3

Conectamos la IA a MySQL

1. Escribí **/clear**.
2. Preguntá: «¿Cuántas unidades hay de cada producto?». Mostrá que adivina.
3. Escribí el prompt de la derecha.
4. Abrí **.mcp.json** y mostralo.
5. Escribí **/mcp**: tiene que decir conectado. Si no: Ctrl+Shift+P › Reload Window.

ESCRIBÍ EN CLAUDE CODE

Conectate a la base MySQL de este proyecto con un servidor MCP y guardalo en la configuración del proyecto.

Clave: .mcp.json queda en el repo, todos tienen la misma conexión. En Antigravity: mcp_config.json.

EN VIVO 3

Ahora pregunta, no inventa

1. Escribí el prompt.
2. Cuando pida usar la herramienta de MySQL, pará y mostrala.
3. Aprobá y mostrá los datos reales que vuelven.
4. Recargá la tienda: aparece el aviso.

ESCRIBÍ EN CLAUDE CODE

¿Qué productos tienen menos de 5 unidades? Proponé un aviso de «últimas unidades» en la tienda y aplicalo.

Pregunta: ¿le darían a un agente acceso libre a la base de producción? Lo resolvemos con hooks.

CONCEPTO 4 · SKILLS

Skills: recetas que aprende una vez.

- ¿Cuántas veces escribieron «validá, usá transacción, probá con curl»?
- Una skill es un archivo: arriba dice cuándo usarla, abajo cómo.
- Sólo se lee cuando hace falta: no ocupa el escritorio.
- Se llama con **/nombre** o la elige el agente. Funciona igual en Antigravity.

```
SKILL.md
name: nuevo-endpoint
description: usar cuando
pidan un endpoint
1. Leer el esquema
2. Validar con Zod
3. Probar con curl
```

EN VIVO 4

La IA escribe su propia receta

1. Escribí **/clear**.
2. Escribí el prompt.
3. Abrí **.claude/skills/nuevo-endpoint/SKILL.md**.
4. Mostrá: arriba name y description, abajo los pasos.
5. Escribí **/** en el panel: aparece la skill.

ESCRIBÍ EN CLAUDE CODE

Creá una skill «nuevo-endpoint» para este proyecto con los pasos para crear un endpoint: leer el esquema, validar con Zod, SQL con parámetros, transacción si toca varias tablas, registrar la ruta y probar con curl un caso válido y uno inválido.

Clave: la description es una condición, «usar cuando pidan un endpoint». Así la elige sola.

EN VIVO 4

Usamos la receta

1. Dejá el SKILL.md abierto al lado.
2. Escribí el prompt.
3. Marcá qué paso de la receta está haciendo.
4. Si se saltea uno: editá el SKILL.md, guardá y repetí. Así se mejora una skill.

ESCRIBÍ EN CLAUDE CODE

/nuevo-endpoint Creá la compra: POST /api/orders que reciba nombre, email y los productos con cantidad, verifique stock, lo descuente y guarde la orden.

¿Por qué transacción? Si descuenta stock y falla al guardar la orden, perdemos stock. Todo o nada.

Pausa · 5 minutos

A la vuelta: un equipo de agentes trabajando al mismo tiempo.

CONCEPTO 5 · SUBAGENTES

Un equipo, no un asistente.

- Un solo agente hace una cosa por vez y se le llena el escritorio.
- Subagentes: cada uno con su especialidad, sus permisos y su propio contexto.
- El agente principal hace de jefe: reparte y junta resultados.
- QA no puede editar código: quien construye no aprueba. Mínimo privilegio.



EN VIVO 5

Contratamos al equipo

1. Escribí **/clear**.
2. Escribí el prompt.
3. Escribí **/agents**: aparecen los cuatro.
4. Abrí el archivo de QA en la carpeta **.claude/agents/**.
5. Mostrá: nombre, descripción, herramientas y modelo.

ESCRIBÍ EN CLAUDE CODE

Creá 4 subagentes para este proyecto: base de datos, backend, frontend y QA. Cada uno sólo con las herramientas que necesita. QA no puede editar código, sólo probar y reportar. Para QA usá el modelo haiku.

Clave: un subagente es un Markdown. QA usa haiku: modelo chico y barato para una tarea repetitiva.

EN VIVO 5

El equipo trabaja en paralelo

1. Escribí el prompt.
2. Señalá cuando aparecen varios subagentes a la vez.
3. Leé el informe de QA.
4. Si encontró un bug, pedí: «Corregí lo que encontró QA y que vuelva a probar».

ESCRIBÍ EN CLAUDE CODE

Implementá el checkout completo: formulario con nombre y email, botón de comprar que llame a POST /api/orders y una pantalla de confirmación. Delegá en los subagentes, en paralelo cuando se pueda. Al final QA prueba todo y me das un informe.

Clave: cada uno trabaja en su propio contexto. Al jefe le llega sólo el resumen.

CONCEPTO 6 · HOOKS

Hooks: reglas que no se negocian.

- Pedirle «por favor» funciona casi siempre. Para borrar una base, casi no alcanza.
- Un hook es código nuestro que se dispara en cada acción del agente.
- Sirve para dos cosas: frenarlo y verlo.
- Prompt injection: lo que la IA lee nunca debería ser una orden.

Frenar
nada de borrar tablas

Ver
avisarle a Jarvis

EN VIVO 6

Le ponemos límites

1. Escribí **/clear**.
2. Escribí el prompt 1.
3. Mostrá **.claude/settings.json** por arriba.
4. Escribí **/hooks**: aparece registrado.
5. Escribí el prompt 2 y mirá cómo lo frena.

PROMPT 1

Agregá un hook de proyecto que, antes de ejecutar cualquier comando, bloquee los que tengan DROP TABLE, DROP DATABASE o rm -rf, y le explique al agente por qué.

PROMPT 2 · LA PRUEBA

Borrá la tabla orders.

Clave: se lo pedí yo, el dueño, y no pudo. La seguridad está afuera del modelo.

EN VIVO 7

Construimos a Jarvis

1. Escribí **/clear**.
2. Escribí el prompt.
3. Pedí: «Levantá el tablero».
4. Abrí **localhost:5055**.
5. Acomodá la pantalla: VS Code a la izquierda (Win + ←), tablero a la derecha (Win + →).

ESCRIBÍ EN CLAUDE CODE

Creá un tablero en ops/mission-control: un servidor Node en el puerto 5055 que reciba los eventos de los hooks y una página con Tailwind que muestre en vivo, como un mapa, cada agente como una tarjeta: encendida cuando trabaja, con la herramienta que usa y el archivo que toca. Agregá los hooks necesarios para mandarle todos los eventos, incluidos los de subagentes.

Clave: los hooks también sirven para ver. Como el tablero de Jarvis en Iron Man.

EN VIVO 7

Jarvis en acción

1. Con el tablero a la vista, escribí el prompt.
2. Señalá las tarjetas que se encienden.
3. Mostrá qué archivo toca cada agente.
4. Al terminar, recargá la tienda y probá el buscador.

ESCRIBÍ EN CLAUDE CODE

Agregá al catálogo un buscador de productos, usando los subagentes.

Clave: usar IA en serio es poder ver qué hace. Y el tablero también lo construyó el agente.

CONCEPTO 7 · REMOTE CONTROL

La IA en tu compu, vos en el celular.

- Lanzás tests o una migración y te tenés que ir. ¿Te quedás mirando la pantalla?
- Un comando: la sesión sigue en tu notebook, con tus archivos, tu base y tus MCP.
- Desde el celular manejás la conversación. Tu código no sale de tu máquina.
- Requiere cuenta claude.ai Pro o superior.



EN VIVO 8

Conectamos el celular

1. Compartí VS Code con el tablero y la tienda al lado.
2. En el panel de Claude escribí **/rc**.
3. Aparecen una URL y un QR.
4. Escaneá el QR con el celular.
5. Mostrá el celular a la cámara: es la misma conversación.

/rc

¿No se lee el QR? App Claude › Code › la sesión del proyecto.

Clave: la sesión corre en mi notebook. Desde el celular sólo manejo la conversación.

EN VIVO 8

Desde el celular, frente a la cámara

1. En el celular escribí el prompt 1 y mostralo a la cámara.
2. Te pregunta botones o menú: respondé desde el celular.
3. Señalá la pantalla: el tablero y VS Code se mueven.
4. Al final pide permiso para el commit: aprobalo desde el celular.
5. Achicá la tienda, sacale foto y mandala con el prompt 2.

PROMPT 1 · DESDE EL CELULAR

Agregá un filtro por categoría arriba de los productos, con las categorías reales de la base. Antes de empezar, preguntame si lo quiero con botones o con un menú desplegable. Cuando termines, guardá los cambios en git con un commit.

PROMPT 2 · CON LA FOTO

Así se ve la tienda en móvil. Mejorá las tarjetas y los botones en pantallas chicas.

Usos reales: tareas largas desde el colectivo, revisar un PR, foto del error en lo de un cliente.

Lo que hicimos hoy

Contexto · le dimos memoria

Skills · le dimos recetas

Hooks · le pusimos límites

MCP · le dimos acceso

Subagentes · le dimos un equipo

Remote Control · lo soltamos de la silla

Programar ahora también es dirigir.

TU TURNO

Agregá una función a la tienda sin tocar el código.

Usá una skill nueva, una herramienta MCP y dos subagentes. Mandá una captura de tu Jarvis. Bonus: hacelo también en Antigravity.

¿Preguntas?

Si algo falla en vivo

Problema	Qué hacer
La tienda tarda o se traba	Abrir la carpeta tiendita-respaldo y seguir desde ahí
MySQL no conecta	En la terminal: docker start tienda-mysql, y pedirle que reintente
MCP no aparece	Ctrl + Shift + P › Reload Window, y escribir /mcp
No usa subagentes	Pedir explícito: «usá el subagente backend y el frontend al mismo tiempo»
Jarvis no recibe eventos	Pedir: «el tablero no recibe eventos, revisá los hooks»
/rc da error	Verificar que la sesión sea con tu cuenta claude.ai y no con una API key
Responde raro	/clear y repetir el pedido: el escritorio estaba lleno
Se borró una tabla	Restaurar el respaldo: docker exec -i tienda-mysql mysql -uroot -proot tienda < respaldo.sql

Si falla, decilo en voz alta: un agente que se equivoca y se corrige también enseña.